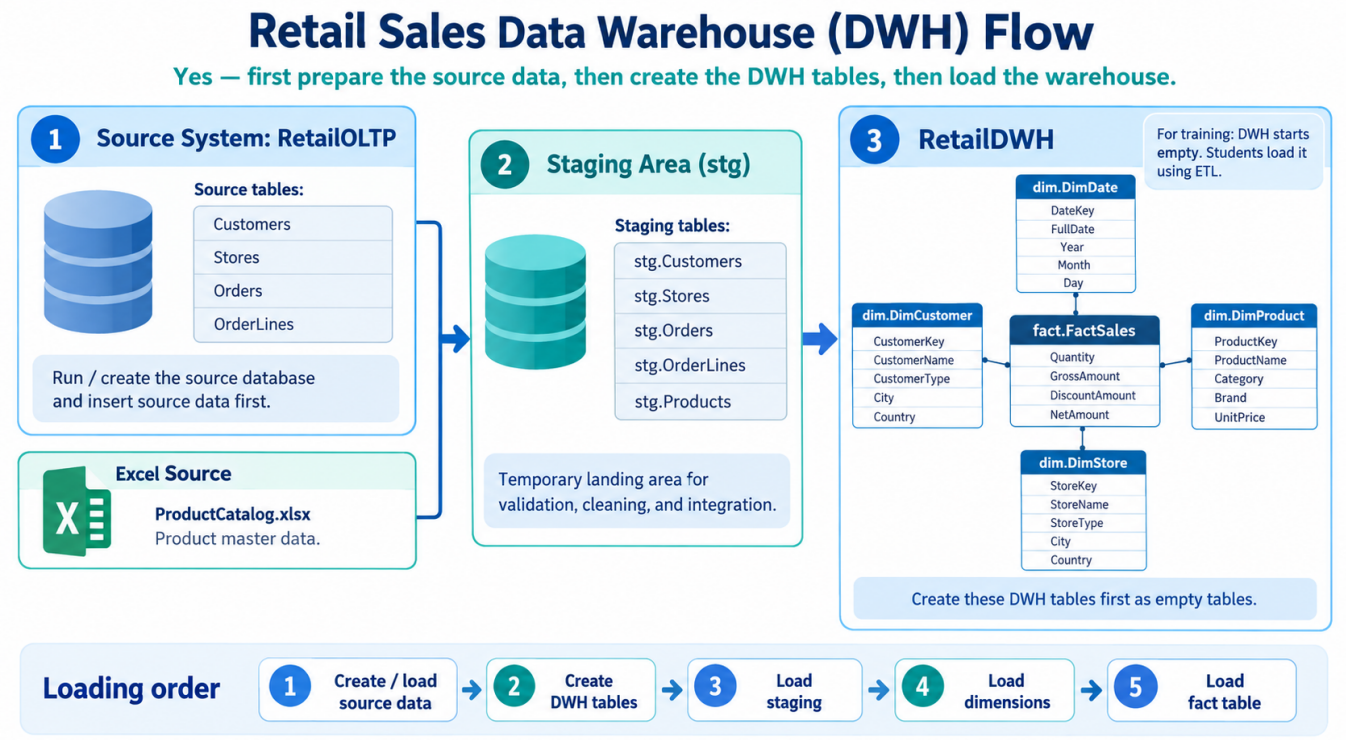
STUDENT PRACTICAL WORKBOOK / MICROSOFT SSIS

Retail Sales Data Warehouse

End-to-end ETL lab guide | SQL Server + SSIS | RetailOLTP → RetailDWH



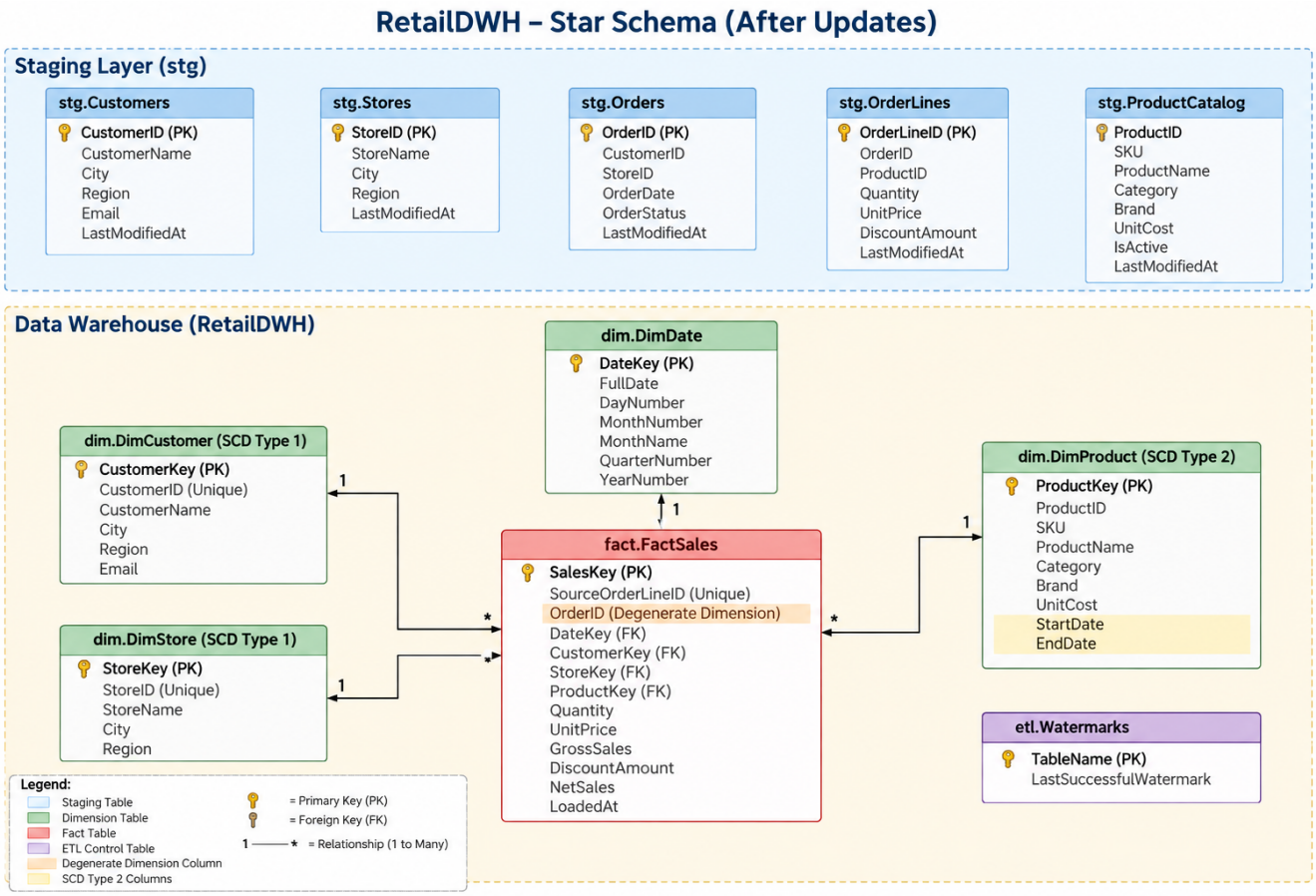
> Lab goal: Build SSIS packages to populate staging, load the four dimensions, resolve surrogate keys, and load valid completed order lines into fact.FactSales. Handle missing lookups and validate every load.

Provided assets: source SQL scripts, DWH schema SQL, Excel/CSV product catalog, architecture diagrams, and an SSIS project ZIP.

Audience: Junior Data Engineers / ETL students. Level: guided hands-on lab.

01 Understand the target design

Read this page before opening Visual Studio.



The diagram above is the detailed target schema. Use the supplied SQL DDL as the authoritative definition when names differ from a simplified architecture illustration.

Three layers, one grain

• RetailOLTP: operational SQL Server source tables dbo.Customers, dbo.Stores, dbo.Orders, and dbo.OrderLines. Product attributes come from the Excel catalog.

• RetailDWH.stg: landing tables for the five source entities. They are intermediate ETL tables, not the final reporting model.

• RetailDWH.dim / fact: four dimensions and one sales fact. The grain of fact.FactSales is one row per OrderLineID.

> Key concept: OrderID stays in the fact as a degenerate dimension; SourceOrderLineID identifies the original source row; SalesKey is SQL Server-generated. The four dimension foreign keys must be resolved before insertion.

02 Prepare the databases and project

A. Run the supplied files in SSMS - in this order

Order	File / action	Expected outcome
1	01_Create_OLTP_Source(2).sql	Creates RetailOLTP and its four source tables.
2	01b_Insert_OLTP_Sample_Data(2).sql	Populates the sample retail transactions.
3	02_Create_DWH_Star_Schema(1).sql	Creates RetailDWH, stg/dim/fact/etl schemas and empty tables.
4	ProductCatalog_Batch1(1).xlsx	Use as the product Excel Source inside SSIS; do not insert it manually.

Safety: The supplied DWH creation script contains DROP TABLE IF EXISTS. Run it only for a fresh lab reset. Running it after ETL has populated the warehouse deletes your loaded data.

B. Visual Studio / SSDT setup

• Open the provided reatailSalesDWH SSIS solution, or create an Integration Services Project. Configure two OLE DB Connection Managers: RetailOLTP and RetailDWH.

• For the product file, configure an Excel Connection Manager targeting worksheet ProductCatalog$. If the Excel provider bitness causes an error, align the installed ACE provider with the SSIS runtime or adjust Run64BitRuntime for debugging.

• Test both SQL connections and preview Excel rows. Confirm the destination is the RetailDWH database, not RetailOLTP.

C. Existing packages in the ZIP

The supplied SSIS project includes staging packages for Customers, Stores, Orders, OrderLines, and ProductCatalog; dimension packages for Customer, Store, and Product; and a fact.dtsx package. A dedicated DimDate package is not included, so students must create or otherwise populate it before loading facts.

03 Build the Control Flow - run in the right order

Create one master package (for example, Master_RetailDWH.dtsx) and orchestrate child packages using Execute Package Task. Use green success precedence constraints so later stages cannot run after a failure.

Phase	Tasks (SSIS)	Why
1. Staging	stg_customer → stg_store → stg_orders → stg_ordelines → stg_producd	Land all source rows and the Excel catalog.
2. Dimensions	DimDate; Dim_Customer; Dim_Stores; Dim_product	Create keys referenced by FactSales.
3. Sales fact	fact.dtsx	Look up Order, Customer, Store, Product and Date; insert valid rows.
4. Quality	Execute SQL Task + Row Count / logging	Reconcile counts, missing keys, duplicate IDs and amounts.

How to build a staging package

Control Flow: optional Execute SQL Task for a fresh staging reload, then a Data Flow Task. Data Flow: OLE DB Source (RetailOLTP) → optional Data Conversion → OLE DB Destination (RetailDWH.stg). For ProductCatalog use Excel Source instead of OLE DB Source.

> SSIS practice: Keep inserts inside the SSIS Data Flow. An Execute SQL Task for staging cleanup or validation is acceptable. Do not rerun an insert-only staging package against a table with an existing primary key without first handling duplicates.

Full refresh versus incremental

- Initial classroom load: full-refresh the staging tables before repopulating them; perform the refresh before dimensions and facts are loaded.
- Advanced repeatable ETL: use LastModifiedAt and etl.Watermarks to capture new/changed source rows; do not truncate historical dimensions or facts.

04 Stage the five source entities

Package	Source	Destination / key
stg_customer.dtsx	RetailOLTP.dbo.Customers	RetailDWH.stg.Customers (CustomerID)
stg_store.dtsx	RetailOLTP.dbo.Stores	RetailDWH.stg.Stores (StoreID)
stg_orders.dtsx	RetailOLTP.dbo.Orders	RetailDWH.stg.Orders (OrderID)
stg_ordelines.dtsx	RetailOLTP.dbo.OrderLines	RetailDWH.stg.OrderLines (OrderLineID)
stg_producd.dtsx	Excel: ProductCatalog$	RetailDWH.stg.ProductCatalog (ProductID is not declared PK)

Student procedure - repeat for each table

- Drag Data Flow Task onto Control Flow and open it.
- Add the correct Source; select the table/sheet and preview its rows.
- Add Data Conversion where Excel types do not match the SQL destination. Check numeric, Unicode and date columns.
- Connect to OLE DB Destination; select the corresponding stg table and inspect the Mappings tab.
- Execute; check green check marks and compare source/destination row counts.

Expected first-load staging counts

Customers	Stores	Orders	OrderLines	Products
30	6	180	470	24

These counts are from the supplied sample SQL and Batch 1 product catalog. Counts change if students modify the input data.

Excel-specific check

The product catalog has ProductID, SKU, ProductName, Category, Brand, UnitCost, IsActive, LastModifiedAt. The IsActive flag is a business attribute in staging; it is not an SCD2 current-row flag.

05 Load dimensions using SSIS

DimCustomer and DimStore - SCD Type 1

• Start from stg.Customers / stg.Stores with an OLE DB Source.

• Configure a Slowly Changing Dimension transformation. Use CustomerID or StoreID as the business key, not the identity surrogate key.

• Classify descriptive fields (for example, City, Region, CustomerName) as Changing Attributes. Type 1 updates overwrite old values.

• Send new members to an OLE DB Destination; route changed members to an update path. SQL Server generates CustomerKey and StoreKey.

DimProduct - SCD Type 2

• Read stg.ProductCatalog. Match on ProductID; set historically tracked attributes (for example, Category, Brand, UnitCost) according to your chosen lab rules.

• When a tracked attribute changes, expire the prior version and insert a new version with a new ProductKey.

• Maintain StartDate and EndDate (DATETIME2(3)). This table has no IsCurrent column; use the end-date convention consistently.

> Important SCD2 caveat: The sample sales occurred in July-September 2026. If you create initial product versions using the current ETL run time as StartDate, those rows will not cover historical OrderDate values. For a production as-of lookup, use meaningful effective dates and enforce non-overlapping product version ranges. For the initial training batch (one version per product), a ProductID-only lookup is a simplification, not a complete historical SCD2 solution.

Verify after dimension load

```
SELECT COUNT(*) AS Customers FROM RetailDWH.dim.DimCustomer;
SELECT COUNT(*) AS Stores    FROM RetailDWH.dim.DimStore;
SELECT COUNT(*) AS Products  FROM RetailDWH.dim.DimProduct;
```

06 Create and load DimDate - before FactSales

dim.DimDate is defined in the supplied DWH SQL script, but it is not populated by that script. It has DateKey INT (YYYYMMDD) and FullDate DATE. Students must load it through SSIS before inserting FactSales.

Simple classroom SSIS implementation

• Create Dim_Date.dtsx with an OLE DB Source connected to RetailOLTP. Use the SQL query below to generate dates and calendar attributes from the sample orders.

```
SELECT DISTINCT
    CONVERT(INT, CONVERT(CHAR(8), OrderDate, 112)) AS DateKey,
    CAST(OrderDate AS DATE) AS FullDate,
    DAY(OrderDate) AS DayNumber,
    MONTH(OrderDate) AS MonthNumber,
    DATENAME(MONTH, OrderDate) AS MonthName,
    DATEPART(QUARTER, OrderDate) AS QuarterNumber,
    YEAR(OrderDate) AS YearNumber
FROM dbo.Orders;
```

• Add an OLE DB Destination mapped to RetailDWH.dim.DimDate. Use a Lookup or another anti-duplicate approach on repeat runs; DateKey and FullDate are unique.

• This lab approach loads dates present in orders only. A real Date dimension should generally cover a continuous calendar range.

DateKey in the FactSales Data Flow

Once OrderDate arrives from the stg.Orders Lookup, use a Derived Column to create an INT key:

```
(DT_I4)(YEAR(OrderDate) * 10000
      + MONTH(OrderDate) * 100
      + DAY(OrderDate))
```

Then add a Lookup against dim.DimDate: connect the input DateKey to reference DateKey, and select the dimension DateKey for validation/output if needed. Do not match OrderDate (datetime) directly to FullDate (date) without converting the types and removing the time portion.

> Example: OrderDate = 2026-09-25 09:29:00 → DateKey = 20260925. The value is both computed from the order date and checked against DimDate.

07 Build FactSales - exact SSIS Data Flow

Create/open fact.dtsx. The OLE DB Source is stg.OrderLines, because one fact row represents one order line. stg.Orders is a Lookup that enriches each line with its header information.

Recommended component order

#	SSIS component	Configuration / result
1	OLE DB Source	stg.OrderLines: OrderLineID, OrderID, ProductID, Quantity, UnitPrice, DiscountAmount
2	Derived Column	GrossSales = Quantity * UnitPrice; NetSales = GrossSales - DiscountAmount
3	Lookup: Orders	Join OrderID → stg.Orders.OrderID; return CustomerID, StoreID, OrderDate, OrderStatus
4	Conditional Split	Keep only OrderStatus == "Completed"; route other statuses away
5	Lookup: Customer	CustomerID → dim.DimCustomer.CustomerID; return CustomerKey
6	Lookup: Store	StoreID → dim.DimStore.StoreID; return StoreKey
7	Lookup: Product	ProductID → dim.DimProduct.ProductID; return ProductKey (initial batch only)
8	Derived Column + Date Lookup	Calculate DateKey; validate against dim.DimDate.DateKey
9	Lookup: existing fact (optional)	OrderLineID → fact.FactSales.SourceOrderLineID; insert only no-match rows
10	OLE DB Destination	Insert mapped columns into fact.FactSales; redirect missing lookups to reject output

SSIS Conditional Split expression

```
OrderStatus == "Completed"
```

SSIS expressions use double quotes for strings. Single quotes or an unquoted Completed value will fail expression parsing.

08 Configure the Lookups correctly

Lookup reference	Input join column	Return / output
stg.Orders	OrderID = OrderID	CustomerID, StoreID, OrderDate, OrderStatus
dim.DimCustomer	CustomerID = CustomerID	CustomerKey
dim.DimStore	StoreID = StoreID	StoreKey
dim.DimProduct	ProductID = ProductID	ProductKey
dim.DimDate	DateKey = DateKey	DateKey (validates date)

Lookup editor: repeat these four actions

- On General, select the cache mode; Full Cache is fine for the small classroom dimensions.
- On Connection, select RetailDWH and the correct stg or dim reference table.
- On Columns, drag the input business key onto the same business key in the reference table. Tick the surrogate key to add to the output.
- On General, select Redirect rows to no match output where you need to inspect and save missing references.

Why a Product lookup can fail

The supplied source includes OrderLineID 50470 with ProductID 9999, while Batch 1 contains products 1001-1024. This is an intentionally unmatched reference. Redirect the row to a Flat File Destination or reject table with the source identifiers and reason. Do not invent a ProductKey.

SCD Type 2: what changes after a second product batch?

If dim.DimProduct has multiple versions for one ProductID, a simple ProductID-only Lookup is ambiguous. The correct historical key is selected using ProductID + effective date range, for example OrderDate >= StartDate AND OrderDate < EndDate for a half-open interval. That temporal logic needs an appropriate SQL reference/query design; an ordinary equality-only SSIS Lookup is not sufficient by itself.

> Classroom scope: For Batch 1, when each ProductID has one dimension row, ProductID → ProductKey is acceptable. Record the SCD2 historical lookup as an extension task.

09 Map the FactSales destination

In OLE DB Destination → Mappings, map each required column. SQL Server generates SalesKey automatically and supplies the LoadedAt default if omitted.

Input SSIS column	Destination: fact.FactSales	Note
OrderLineID	SourceOrderLineID	Required, unique; not nullable
OrderID	OrderID	Degenerate dimension
DateKey	DateKey	Must exist in dim.DimDate
CustomerKey	CustomerKey	Returned from DimCustomer Lookup
StoreKey	StoreKey	Returned from DimStore Lookup
ProductKey	ProductKey	Returned from DimProduct Lookup
Quantity	Quantity	From stg.OrderLines
UnitPrice	UnitPrice	From stg.OrderLines
GrossSales	GrossSales	Quantity × UnitPrice
DiscountAmount	DiscountAmount	From stg.OrderLines
NetSales	NetSales	GrossSales - DiscountAmount
	SalesKey, LoadedAt	IDENTITY / DEFAULT handled by SQL Server

Common failure: "Cannot insert NULL into SourceOrderLineID" means the source OrderLineID was not mapped correctly to SourceOrderLineID. Fix it in Destination → Mappings; do not make the column nullable.

Avoid duplicates when the package is rerun

Because SourceOrderLineID is UNIQUE, an insert-only fact load will fail if you run it twice. For an SSIS-only training solution, add a Lookup against fact.FactSales on OrderLineID = SourceOrderLineID, and send No Match Output to the destination. For changed order lines, design an explicit update path.

10 Validate results in SSMS

Expected results for the supplied first batch

Check	Expected	Explanation
Source OrderLines	470	All source line rows
Completed order lines	431	Lines whose order header status is Completed
Rejected missing product	1	OrderLineID 50470, ProductID 9999
Inserted fact rows	430	431 completed lines - 1 missing product
Other status lines	39	Pending/Cancelled lines excluded from sales fact

These are expected when the dimensions and DimDate are populated, the Completed filter is applied, the unmatched product is rejected, and the fact starts empty.

Verification SQL - counts and business rules

```
USE RetailDWH;
SELECT COUNT(*) AS FactRows FROM fact.FactSales;
SELECT COUNT(*) AS MissingLineIDs
FROM fact.FactSales WHERE SourceOrderLineID IS NULL;
SELECT SourceOrderLineID, COUNT(*) AS Cnt
FROM fact.FactSales
GROUP BY SourceOrderLineID HAVING COUNT(*) > 1;
SELECT COUNT(*) AS InvalidAmounts
FROM fact.FactSales
WHERE ABS(NetSales - (GrossSales - DiscountAmount)) > 0.01;
```

Compare to completed source order lines

```
SELECT COUNT(*) AS CompletedSourceLines
FROM RetailDWH.stg.OrderLines AS l
JOIN RetailDWH.stg.Orders AS o ON o.OrderID = l.OrderID
WHERE o.OrderStatus = 'Completed';
```

Reconciliation: CompletedSourceLines (431) = LoadedFactRows (430) + ProductLookupRejects (1). Save this evidence in your submission.

11 Troubleshooting and next steps

Symptom	Likely cause / action
Lookup: Row yielded no match	Check join key, source values and loaded dimension; redirect no-match rows and log them.
NULL SourceOrderLineID	Map OrderLineID → SourceOrderLineID in the OLE DB Destination.
Date Lookup returns no match	Confirm DimDate has rows; derive YYYYMMDD INT and join DateKey to DateKey.
Conditional Split parsing error	Use OrderStatus == "Completed" (double quotes).
Duplicate key / unique violation	Use a target Lookup or incremental upsert; do not reinsert the same OrderLineID.
Excel provider / bitness error	Verify ACE installation and SSIS runtime architecture; check worksheet and data types.
Foreign key insert failure	Load dimensions first; ensure every fact FK points to an existing dimension row.

Extension - implement incremental loads

- Use etl.Watermarks(TableName, LastSuccessfulWatermark) to store the last successful load timestamp for each source entity.
- For repeat runs, extract rows with LastModifiedAt newer than the stored watermark, then upsert staging/dimension/fact records as appropriate.
- Advance the watermark only after successful ETL. For FactSales, consider both changed order lines and changed order headers, because an updated OrderStatus affects which lines qualify.
- Do not truncate dim.DimProduct (SCD2 history) or fact.FactSales as part of normal incremental runs.

12 Student assignment and submission checklist

Complete the tasks below in your own SSIS project. The provided project ZIP may be used as a reference, but you must be able to explain each component and mapping.

☐ 01. Source and warehouse databases created successfully from the supplied scripts.

☐ 02. Five staging Data Flows built and validated (30 / 6 / 180 / 470 / 24 rows).

☐ 03. DimCustomer and DimStore implemented as SCD Type 1.

☐ 04. DimProduct implemented as SCD Type 2 with StartDate and EndDate.

☐ 05. DimDate populated and DateKey verified in YYYYMMDD format.

☐ 06. FactSales sourced from stg.OrderLines and enriched from stg.Orders.

☐ 07. Conditional Split filters only Completed orders.

☐ 08. Customer, Store, Product, and Date Lookups configured correctly.

☐ 09. OrderLineID mapped to SourceOrderLineID; identity columns left unmapped.

☐ 10. Missing ProductID 9999 redirected to an auditable reject output.

☐ 11. Fact row count reconciled: 430 valid rows in the initial lab load.

☐ 12. Screenshots / SQL results saved to demonstrate successful execution.

What to submit

- SSIS solution/project folder (including the master package, staging packages, dimension packages, DimDate, and FactSales).
- Screenshots of Control Flow, FactSales Data Flow, Lookup Columns, and Destination Mappings.
- One screenshot of the reject output and a SQL verification result showing 430 inserted fact rows.
- A short note explaining SCD Type 1 vs Type 2, the purpose of OrderID as a degenerate dimension, and why SourceOrderLineID must be unique.

> Success criteria: A reproducible SSIS ETL workflow that populates the RetailDWH star schema, rejects invalid references, preserves required keys, and produces reconciled sales measures.

Source material: user-provided RetailOLTP SQL, RetailDWH schema SQL, ProductCatalog Batch 1, architecture diagrams, and reatailSalesDWH SSIS project.